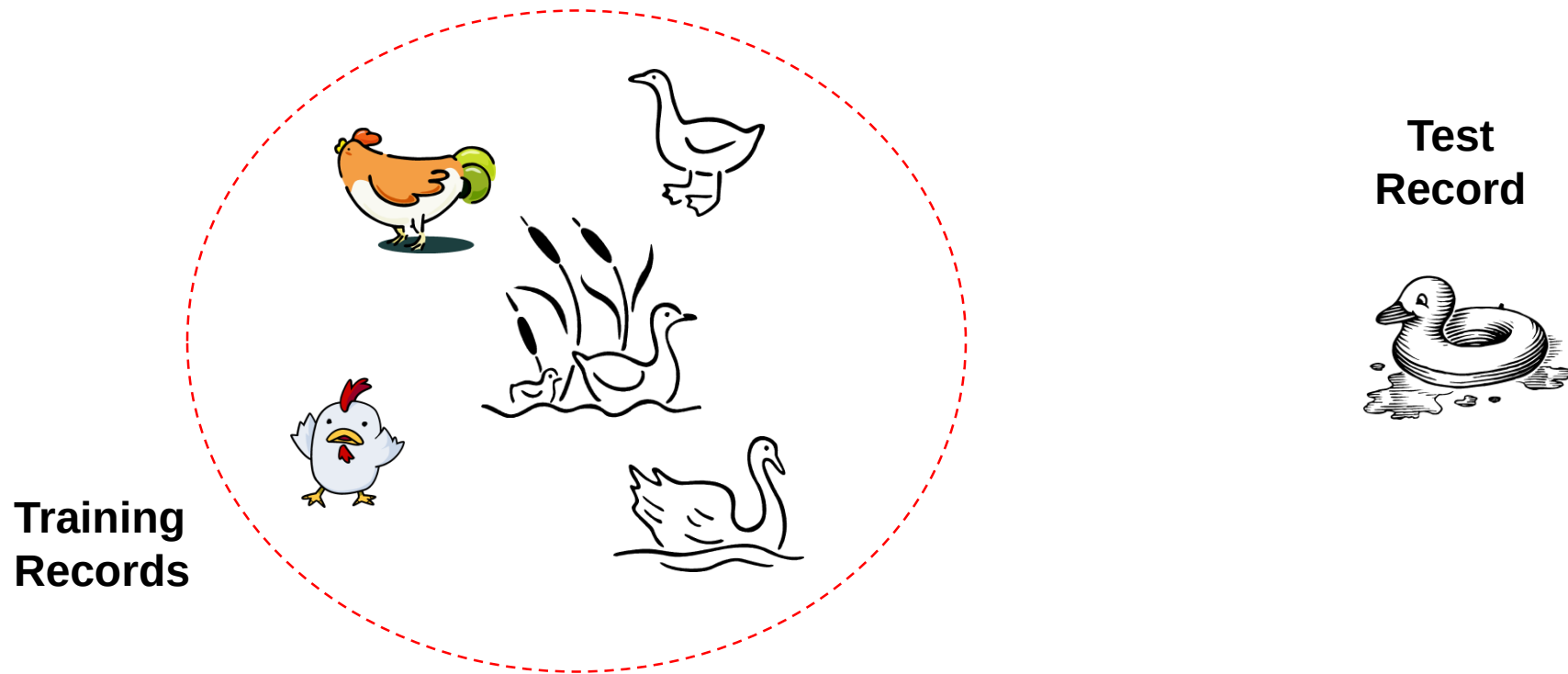

Lecture 15: Nearest Neighbour Classifier

Md. Shahriar Hussain
ECE Department, NSU

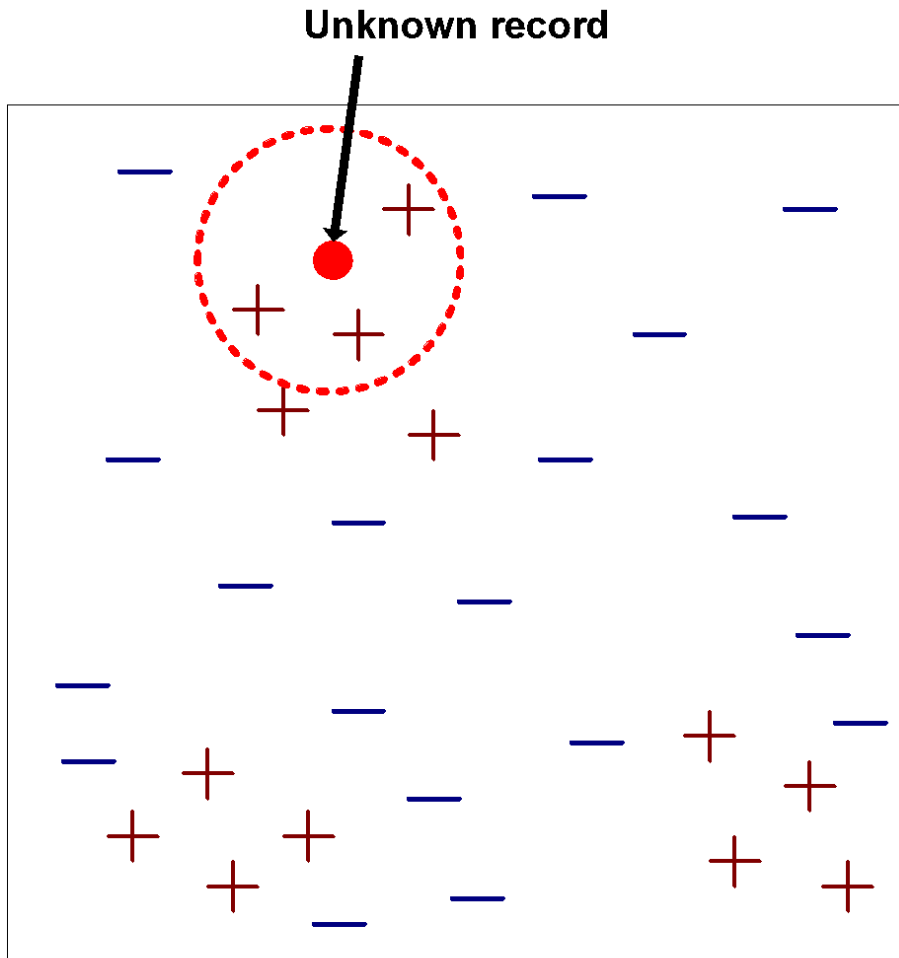


Nearest Neighbor Classifiers

- Basic idea:
 - If it walks like a duck, quacks like a duck, then it's probably a duck



Nearest-Neighbor Classifiers



- Requires the following:
 - A set of labeled records (supervised learning)
 - Proximity metric to compute distance/similarity between a pair of records
 - e.g., Euclidean distance, Manhattan distance
 - The value of k , the number of nearest neighbors to retrieve: how many neighbors you want to consider
 - A method for using class labels of K nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Query Types

- **Exact match query:** Asks for the object(s) whose key matches query key exactly.
- **Range query:** Asks for the objects whose key lies in a specified query range (interval).
- **Nearest-neighbor query:** Asks for the objects whose key is “close” to query key.



Exact Match Query

- Suppose that we store employee records in a database:
- Asks for the object(s) whose key matches query key exactly
- Example:
 - **key=ID:** retrieve the record with ID=12345

ID	Name	Age	Salary	No of Children
----	------	-----	--------	----------------



Range Query

- Example:
 - **key=Age:** retrieve all records satisfying
 $20 < \text{Age} < 50$
 - **key= #Children:** retrieve all records satisfying
 $1 < \# \text{Children} < 4$

ID	Name	Age	Salary	No of Children
----	------	-----	--------	----------------



Nearest-Neighbor(s) (NN) Query

- Example:
 - **key=Salary:** retrieve the employee whose salary is closest to \$50,000 (i.e., 1-NN).
 - **key=Age:** retrieve the 5 employees whose age is closest to 40 (i.e., k-NN, k=5).

ID	Name	Age	Salary	No of Children
----	------	-----	--------	----------------



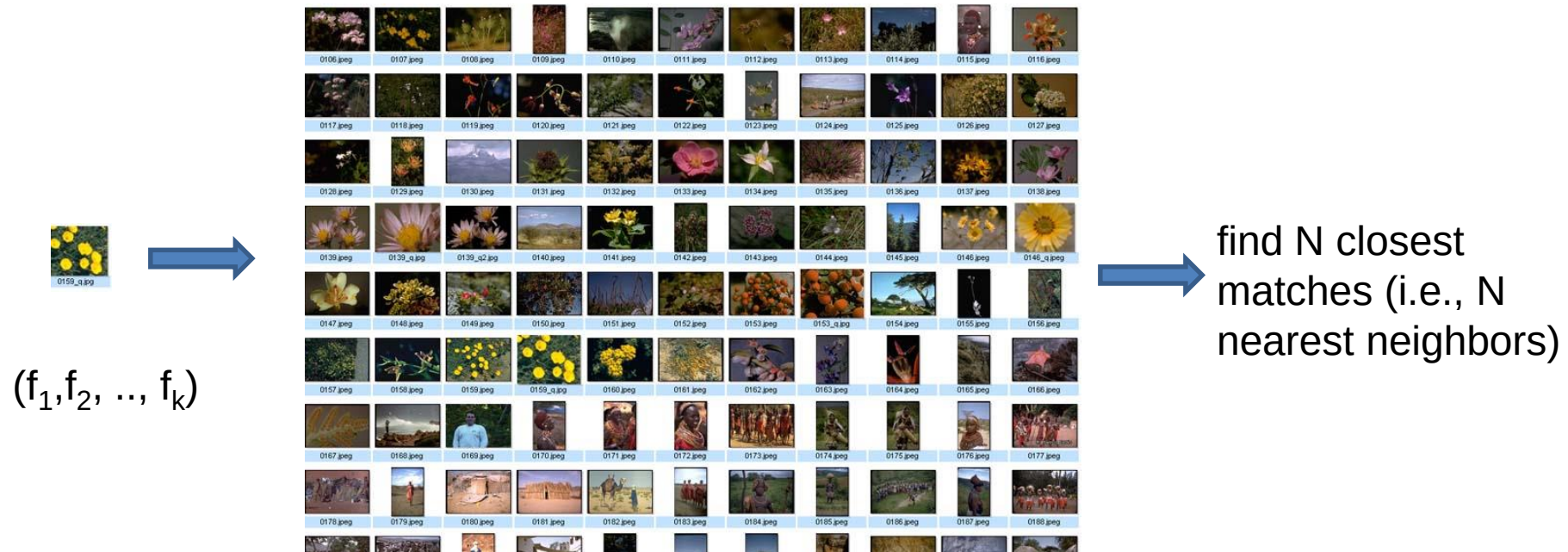
Multi-dimensional Query

- In practice, queries might involve multi-dimensional keys.
 - **key=(Name, Age):** retrieve all records with
Name="George" and "50 <= Age <= 70"



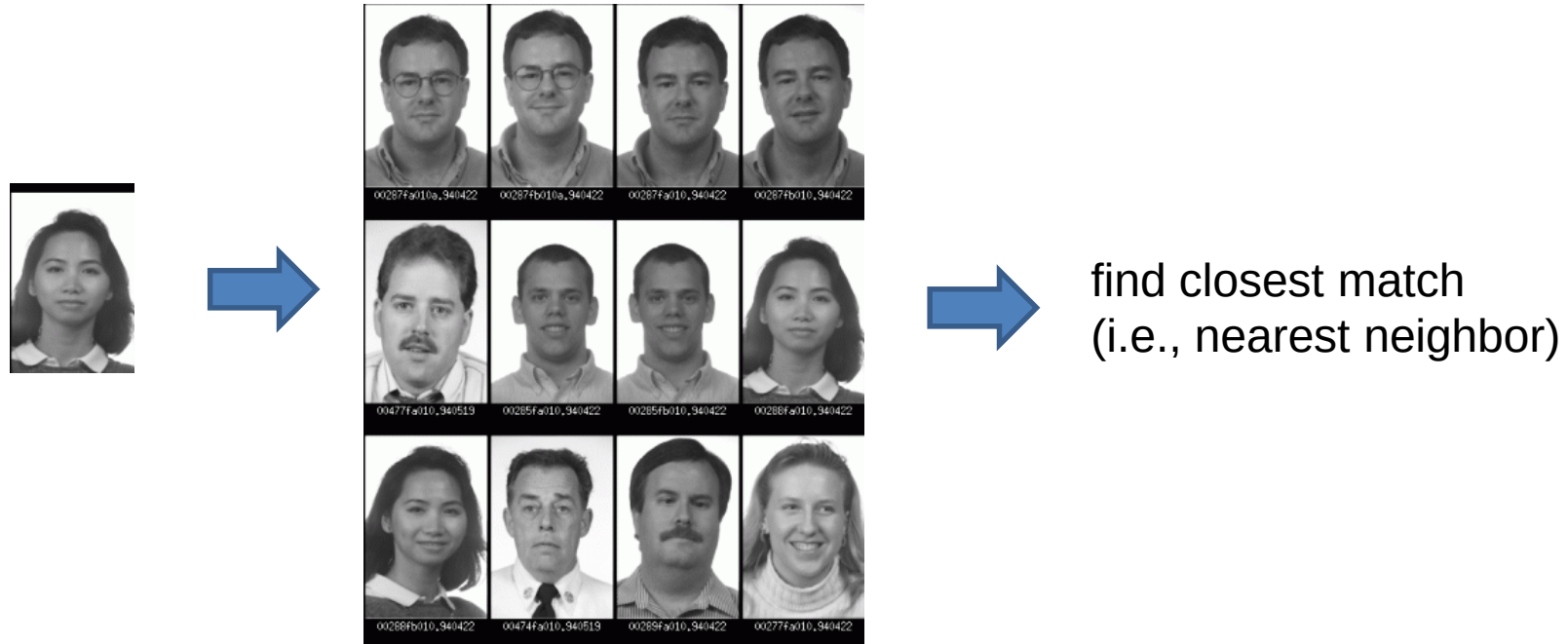
Nearest Neighbor Query in High Dimensions

- Very important and practical problem!
 - Image retrieval



Nearest Neighbor Query in High Dimensions

- Face recognition



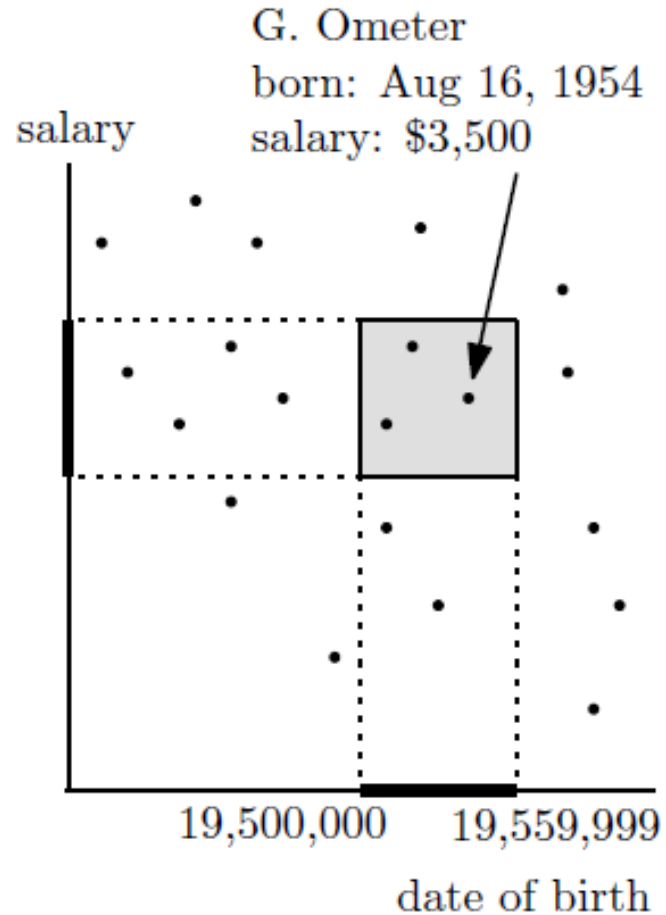
Interpreting Queries Geometrically

- Multi-dimensional keys can be thought as “points” in high dimensional spaces
- Queries about records → Queries about points



Example 1- Range Search in 2D

A database query may ask for all employees with age between a_1 and a_2 , and salary between s_1 and s_2

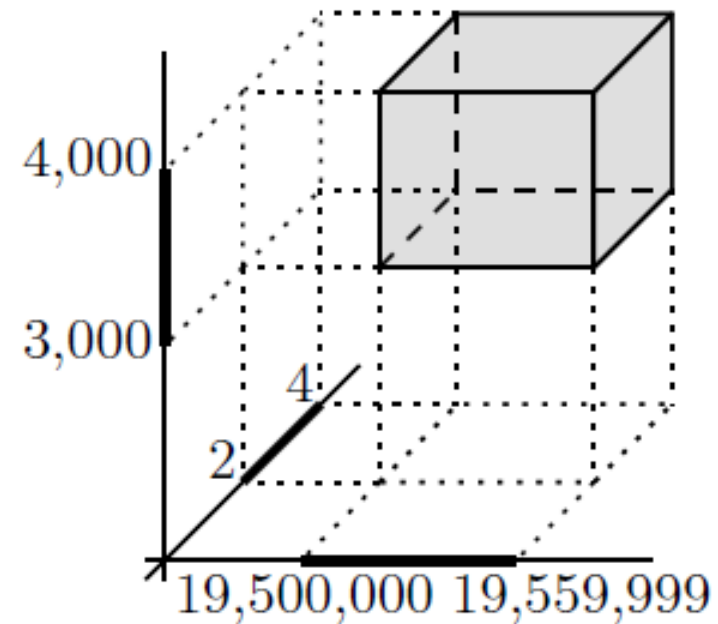


$$\text{age} = 10,000 \times \text{year} + 100 \times \text{month} + \text{day}$$

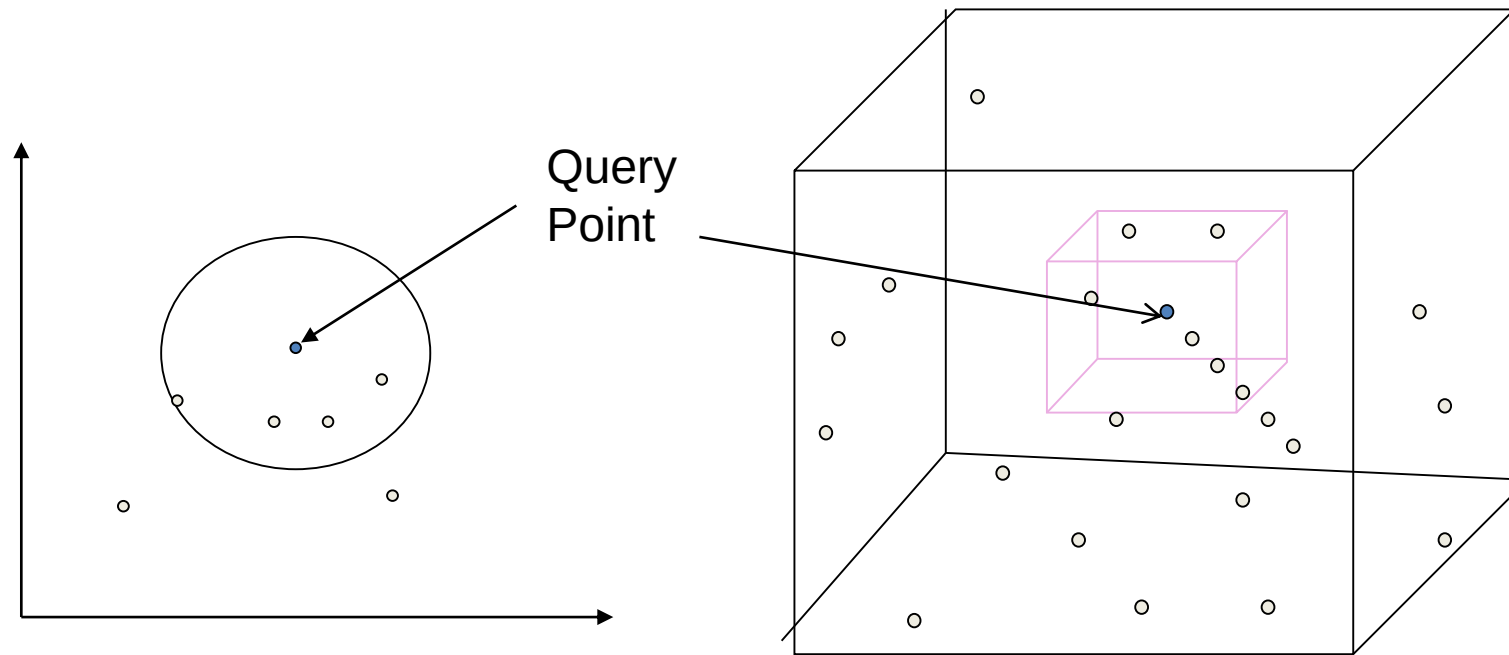


Example 2 – Range Search in 3D

Example of a 3-dimensional (orthogonal) range query:
children in $[2, 4]$, salary in $[3000, 4000]$, date of birth in $[19,500,000, 19,559,999]$



Example 3 – Nearest Neighbors Search



Measure point-to-point distance.

Nearest Neighbor Classification

- **Data preprocessing is often required**

- Attributes may have to be **scaled** to prevent distance measures from being dominated by one of the attributes

- ◆ **Example:**

- height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M
-
- Time series are often standardized to have 0 means a standard deviation of 1



Nearest Neighbor Classification

- If K = 3: Predicted class: Y (majority/unweighted)

Age	Loan	Default	Distance	
25	\$40,000	N	102000	
35	\$60,000	N	82000	
45	\$80,000	N	62000	
20	\$20,000	N	122000	
35	\$120,000	N	22000	2
52	\$18,000	N	124000	
23	\$95,000	Y	47000	
40	\$62,000	Y	80000	
60	\$100,000	Y	42000	3
48	\$220,000	Y	78000	
33	\$150,000	Y	8000	1
48	\$142,000	?		

Euclidean Distance

$$D = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$



K-Nearest Neighbor (kNN)

- kNN is a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset
- **Manhattan distance:** Manhattan Distance is the sum of absolute differences between points across all the dimensions

$$D_m = \sum_{i=1}^n |p_i - q_i|$$



K-Nearest Neighbor (kNN)

- **Minkowski Distance** is the generalized form of Euclidean and Manhattan Distance

$$D = \left(\sum_{i=1}^n |p_i - q_i|^p \right)^{1/p}$$

- When the order is 1, both Minkowski and Manhattan Distance are the same
- When the order is 2, we can see that Minkowski and Euclidean distances are the same



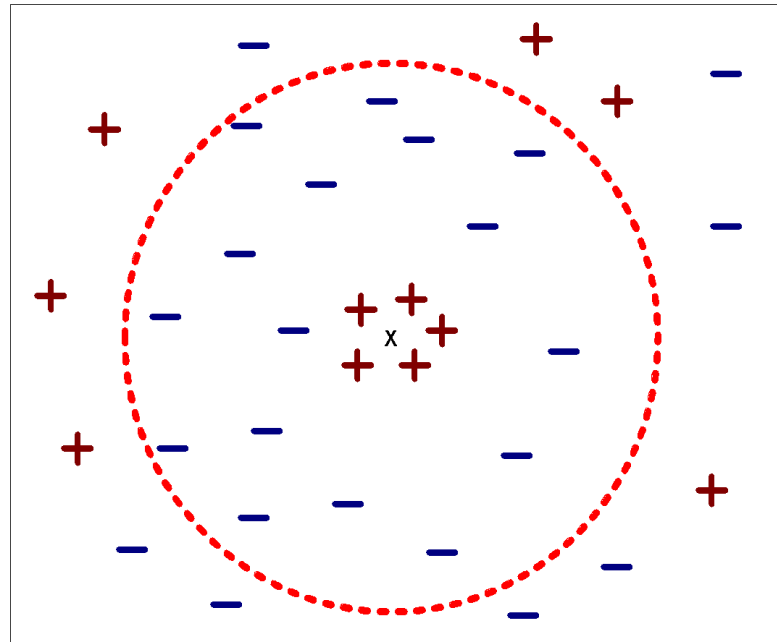
Need for Standardizing Attributes: Feature Scaling

- Feature Scaling is a technique to standardize the independent features present in the data in a fixed range
- If Feature Scaling is NOT done: LOAN feature will dominate all other features while predicting the class of the given data point
- $X_s = \frac{X_i - \text{Min}}{\text{Max} - \text{Min}}$: **Min-Max scaling** - This technique re-scales a feature between 0 and 1
- $X_s = \frac{X_i - \mu}{\sigma}$: **Standardization**
- Standardization is the process of scaling data so that they have a mean value of 0 and a standard deviation of 1
- Good for normally distributed features
- MinMaxScaler and StandardScaler: sklearn



Nearest Neighbor Classification

- Choosing the value of k :
 - Min $k = 1$; Max $k = \#$ of samples
 - If k is too small, sensitive/vulnerable to noise points/outlier
 - If k is too large, neighborhood may include points from other classes: ZeroR classifier



kNN sklearn and hyperparameters

`sklearn.neighbors.KNeighborsClassifier`

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, *,  
      weights='uniform', algorithm='auto', leaf_size=30, p=2,  
      metric='minkowski', metric_params=None, n_jobs=None)
```

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away
- `algorithm`{'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'
- Power parameter for the Minkowski metric. When $p = 1$, this is equivalent to using `manhattan_distance` (l_1), and `euclidean_distance` (l_2) for $p = 2$. For arbitrary p , `minkowski_distance` (l_p) is used

